



cranelift

C++ → WASM → Clif → Asm/Elf

code block 1

code block 2

cranelift

C++ → WASM → Clif → Asm/Elf

code block 1

sub-block

code block 2

Wasm Migration: (1)

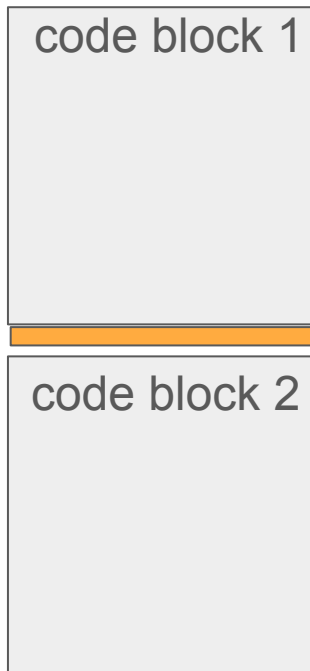
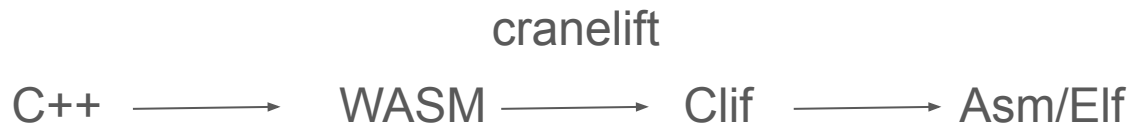


code block 1

code block 2

static code analysis
to find the blocks

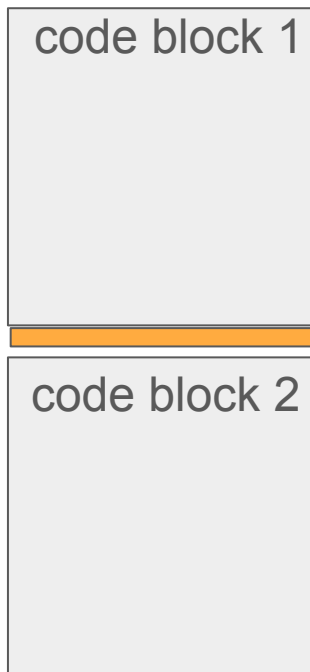
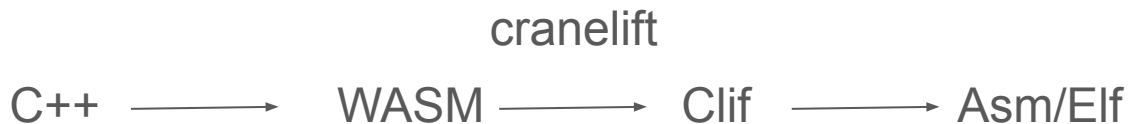
Wasm Migration: (2)



Inject the (save/skip) code

```
If start_from > curr_mem+size(block2)
    jump to next block
else
    if migration
        save ACTIVE memory
        terminate execution
```

Wasm Migration: (2)

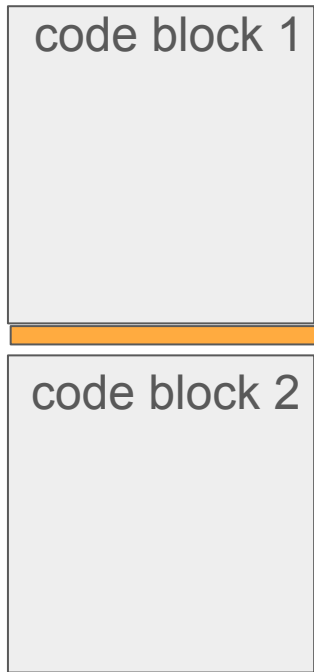
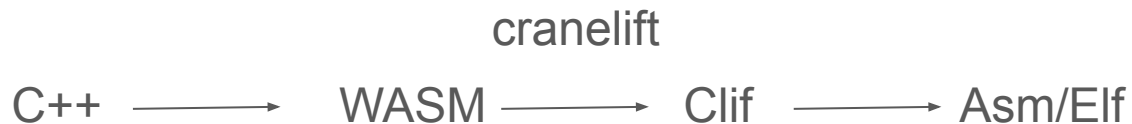


Inject the (save/skip) code

```
If start_from > curr_mem+size(block2)
    jump to next block
else
    if migration
        save ACTIVE memory
        terminate execution
```

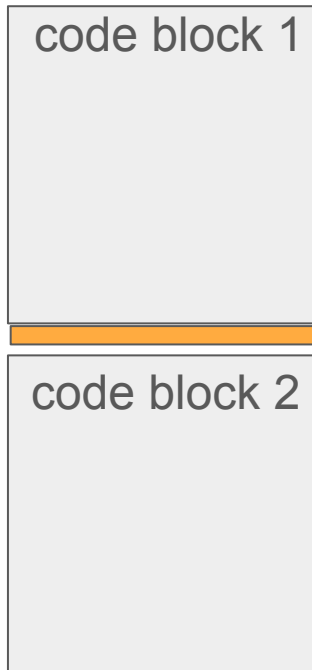
thanks to static analysis we know what the active region of the stack is and what to extract.

Wasm Migration: (3)



When we receive a migration command, we set the "migration" flag to true

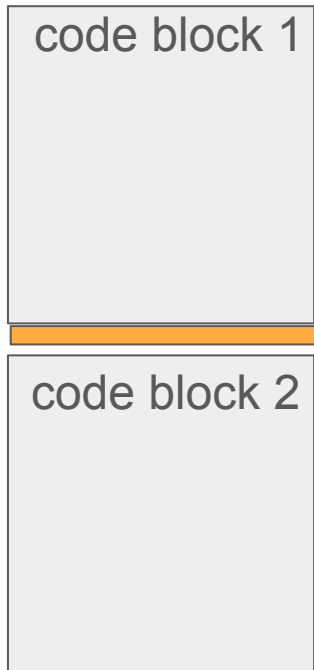
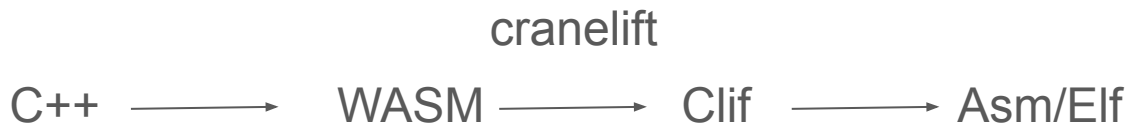
Wasm Migration: (3)



When we receive a migration command, we set the "migration" flag to true

The code will execute until the next checkpoint and give back the active stack references to the runtime

Wasm Migration: (4)



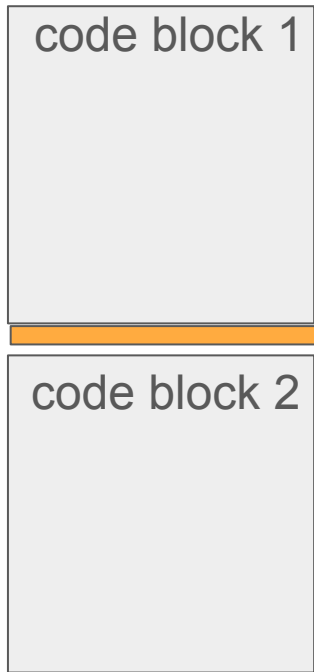
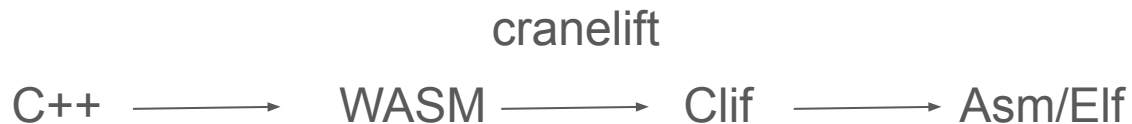
This is where we kick in!

The orchestrator moves the code, the memory, "the network" to the new machine.

sets the `start_from` and starts the application

(Seq diagram later on)

Wasm Migration: (5)



The code will skip all the previous blocks until reaches the resume point.